

whisper-si: A Bit-Exact Integer Whisper-tiny Accelerator

Generated in Chisel with Weights in the Source Tree

Rachit Tibrewal

github.com/tibrewalrachit/wispr-on-chip

Abstract—We present *whisper-si*, a complete hardware implementation of OpenAI’s Whisper-tiny automatic speech recogniser (37.2M parameters) written as a Chisel generator. The design executes the whole model on one chip: a 32×32 int8 weight-stationary matmul engine, a 16-lane vector unit, an attention unit with online softmax over 64-key tiles, a KV cache, a streaming argmax sampler, and a sequencer that executes a 186-instruction micro-program generated from the model graph. All 247 weight tensors are committed to the repository as hex text (38.5 MB) and consumed at elaboration time through one of three interchangeable ROM/SRAM backends. Correctness is defined by an integer golden model in numpy: the RTL is correct if and only if it reproduces the same bits. We show that this holds on 35 full-context Verilator runs over 24 real audio clips, with every token identical to the golden model, and that the integer model meets a word-error-rate gate against the fp32 reference (5.52% vs. 5.54% on 200 LibriSpeech utterances; 7.58% vs. 8.08% on the 20 utterances run on the RTL). An analytic cycle model built only from unit-level measurements predicts every end-to-end run within 0.2%. Logic synthesis of the compute blocks on the ASAP7 7nm predictive PDK gives 0.28mm^2 of standard cells against an estimated $10\text{--}13\text{mm}^2$ of memory; the array closes at 0.79 GHz, the whole design at 0.17 GHz as written, and we identify the two single-cycle arithmetic chains that hold it there and price their pipelining at 8% of cycles. At 1 GHz the measured cycle counts correspond to a 30s window in 42ms plus 0.2ms per generated token.

I. INTRODUCTION

Whisper [1] is the de facto open speech recogniser, and its smallest variant, Whisper-tiny, is an attractive target for a dedicated accelerator: 4 encoder and 4 decoder layers of width 384 with 6 heads, small enough that the entire weight set can live on chip. Building such an accelerator raises three questions that this paper answers with a working design rather than a proposal. (i) Which integer numerics let an int8 datapath match the fp32 model’s accuracy? (ii) How can a Verilator simulation of the chip be checked against real audio without any tolerance? (iii) What does the resulting design cost in cycles and, on a 7nm process, in area and clock rate?

Our contributions are:

- A frozen integer specification of Whisper-tiny (Section III) whose executable form, a numpy model, is both the accuracy vehicle and the source of every RTL test vector.
- A Chisel generator (Section IV) in which the model graph is compiled to a micro-program at generation time, and in which weights are ordinary source files hashed and consumed at elaboration.

- A verification ladder (Section V) from single ops to whole clips in which every level is bit-exact, plus a breakpoint mechanism that locates chip-level divergences by binary search over the micro-program.
- Measured performance (Section VI): cycle counts, an analytic model that reproduces them, and ASAP7 7nm synthesis of every compute block against a 1 GHz target, including the blocks that miss it and what they would cost to fix.

The complete source, weights, references and status reports are in the repository; every number in this paper is reproducible with the `Makefile` targets listed in Section IX.

II. METHOD: THE GOLDEN MODEL IS THE SPECIFICATION

The project ran in five phases, each ending with a commit and a dated status entry; design choices that deviate from the original brief are numbered decisions in `docs/decisions.md`. The central rule is that numerics are frozen in Python before any RTL exists. `golden/whisper_int.py` is a complete integer implementation of Whisper-tiny: it takes an int8 log-mel spectrogram and produces token ids using only integer multiplies, shifts, saturations and table lookups. The only floating-point code is the weight generator, which turns the fp16 checkpoint plus calibration statistics into int8 weights, int32 multipliers and shift amounts.

Because the golden model is executable it doubles as the vector generator: every RTL unit test loads tensors dumped by it, and the end-to-end test compares the RTL token stream with the golden token stream. Accuracy is checked once, at the golden level, against openai-whisper’s own greedy decoder run on the CPU over a single 30s window; the fp32 references regenerate byte-identically and agree with the HuggingFace weights to zero difference on 60 clips. The gate is that the integer model’s WER may exceed fp32 by at most 0.5 points absolute on 200 LibriSpeech test-clean utterances [2]; it passes at 5.52% versus 5.54%.

III. NUMERICS

Throughout, $\text{rsr}(x, s) = (x + 2^{s-1}) \gg s$ and sat_W clamps to W -bit two’s complement; every rounding and saturation is explicit and there is no tolerance anywhere. Table I lists the tensor formats.

TABLE I
ACTIVATION FORMATS. “STATIC” SCALES COME FROM CALIBRATION ON 128 DEV-CLEAN UTTERANCES; “DYNAMIC” SCALES ARE PER TOKEN.

Tensor	Format	Scale
mel input	int8	static
conv outputs, residual stream	int16	static per point
LayerNorm output	int16, 2^{-F}	transient
matmul inputs	int8	dynamic per token
Q, K	int8	static per head
V	int8	static per tensor
attention output	int16	$O/l \cdot 2^7$
fc1 output (GELU input)	int16	static
LM-head logits	int32	wide mode

a) *Matmul inputs and edge requantisation*: Weights are int8, symmetric, per output channel. Matmul inputs are int8 with a per-token dynamic scale: for each row the vector unit finds $m = \max |x_{16}|$, computes $r = \lfloor (127 \cdot 2^{16} + m/2)/m \rfloor$ and quantises $a_8 = \text{sat}_8(\text{rsr}(x_{16} r, 16))$. The row’s m travels with the tile as a *row factor* so the matmul drain can undo it:

$$t = \text{sat}_{40}(\text{rsr}(\text{acc} \cdot M[n], s_1)), \quad (1)$$

$$y = \text{sat}_W(\text{rsr}(t \cdot \text{rowfac}[m] + (B[n] \ll (s_2 - 16)), s_2)), \quad (2)$$

with $s_2 = 24$ for int8 and 20 for int16 outputs. A wide mode keeps int32 logits for the LM head.

b) *Residual stream*: Calibration shows encoder residual maxima of 274 against channel medians of 5.8 (Whisper’s well-known outlier channels [3]). An int8 residual leaves typical values 0–2 LSBs and yields 98% WER, so the residual is int16 (decision 5); int8 remains a configuration option.

c) *LayerNorm*: uses an integer mean $\text{rsr}(\sum x \cdot 5592405, 23)$ (i.e. $\sum x \cdot 2^8/384$), a variance from $\sum x^2$, a 256-entry inverse-square-root table on the normalised mantissa followed by one Newton step, and per-channel gain and bias folded into the output scale.

d) *GELU*: is computed as $x \cdot \Phi(x)$ on the int16 fc1 output with a 256-entry Φ table and linear interpolation (decision 6). The alternative int8→int8 table, with fc1 maxima of 326 against medians of 14, costs 21% WER.

e) *Attention*: Q and K use static per-head scales and V a per-tensor scale, since per-token scales cannot be factored out of the softmax. Scores $s = q_8 \cdot k_8$ are processed in key tiles of 64 with an online softmax [5]: a running maximum m_x , distances $d = \min(4095, \text{rsr}((m_x - s)M_q, s_q))$, a Q1.15 exponential table $E(d)$ with 16-point interpolation, a rescale by $\alpha = E(\Delta m)$ when the maximum grows, and a 36-bit accumulator $O += \sum_j p_j v_j$. After the last tile a pipelined divider forms $r_l = \lfloor (2^{40} + l/2)/l \rfloor$ and the output is $\text{sat}_{16}(\text{rsr}(O r_l, 33))$.

f) *Decoding*: replicates Whisper’s greedy rules exactly: a four-token prompt, sampling from position 3, suppression of 88 non-speech tokens (plus blank and end-of-text on the first step), lowest-index argmax, and a stop at end-of-text or 224 tokens.

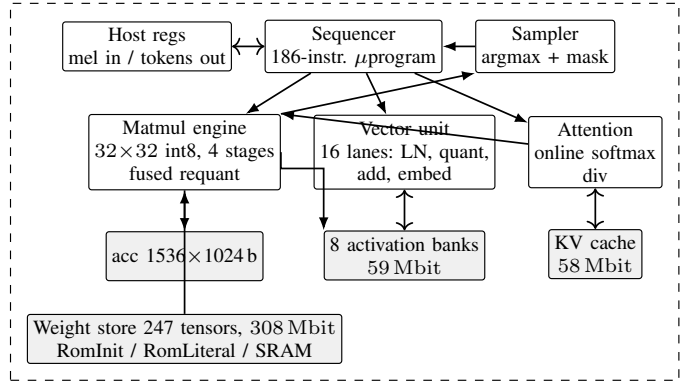


Fig. 1. WhisperTop. The attention unit and the sampler are clients of the single matmul engine; the KV cache feeds the engine’s weight port directly.

IV. ARCHITECTURE

Fig. 1 shows the top level. The host writes the language token and frame count, streams int8 mel rows, and sets a start bit; tokens appear on an output stream with a 256-deep queue. Breakpoint and resume registers pause the sequencer at any micro-instruction so a test can read the on-chip memories through a debug port.

A. Matmul engine

The engine is a 32×32 array of int8 multiply-accumulate cells, weight-stationary and output-tiled: for each output column tile it walks the reduction tiles, loads a 32×32 weight tile in four cycles from a 2048-bit ROM word (one word is one 8-row tile group), then streams all M rows of activations through it, accumulating into a 1536×32 int32 accumulator. The array is organised as four pipeline stages of eight combinational MAC rows (decision 9): a fully skewed systolic array would need either a 2048-bit, 32-stage weight-load chain or a 32-cycle tile switch, whereas eight rows per stage matches the ROM word to one stage and keeps a five-cycle latency. Activations broadcast along rows and partial sums flow down; a reset-initialised valid chain guarantees no spurious output beats. The drain performs the fused requantisation of Section III and routes each 32-wide output row to one of four sinks: an activation bank, the attention unit, the sampler or the KV cache. The per-job cost is $K_T N_T (M + 4) + 10$ cycles, i.e. 99.7% utilisation at $M = 1500$ and five cycles per tile at $M = 1$, the decoder case, where the design is weight-port bound by construction (a 32-byte port would give 1/32 utilisation, decision 8).

B. Vector unit

A 16-lane unit executes LayerNorm, dynamic quantisation, scaled adds and embedding extraction one row at a time in three passes over a 384- or 1536-wide row, optionally fusing a GELU on its input. Measured costs per row are 123 cycles for LayerNorm, 85 for dynamic quantisation, 229 for GELU plus quantisation over 1536 columns and 30–33 for a scaled add. The row orientation is what lets the encoder run every op over 1500 rows with nothing wider than one row in registers.

TABLE II
THE VERIFICATION LADDER. EVERY LEVEL IS BIT-EXACT AGAINST THE GOLDEN MODEL.

Level	Compared	Result
Ops	each fixed-point op vs. float	18 tests pass
Golden vs. fp32	WER, 200 utterances	5.52 vs. 5.54 %
Matmul engine	19 cases, all real shapes	bit-exact
Weight backends	3 backends, real tensors	identical, 13/13
Vector, attention, KV	14 cases from dumps	bit-exact
Full chip, Verilator	35 runs, 24 clips	tokens identical 35/35
RTL WER	20 utterances	7.58 vs. fp32 8.08 %

C. Attention and KV cache

The attention unit owns no multipliers: it drives the engine. For each head and 64-query block it issues, per 64-key tile, one $S = QK^T$ job with K read from the cache as the stationary tile, runs the online softmax over the 64 scores of each query in 12 cycles, then issues two PV jobs in which the 16-bit probabilities are split into unsigned high and low bytes so the int8 array is reused, the results being recombined exactly. After the last key tile a 41-bit pipelined divider finalises each query row in 31 cycles. The cache stores K transposed (one word = 8 head-dims $\times$ 32 keys) and V key-major (8 keys $\times$ 32 dims) so both feed the weight port directly; a double-buffered transposer accepts engine rows without stalling and byte-masked writes append single keys while decoding.

D. Sequencer and micro-program

The model graph is not hard-wired. A generator walks the layer structure and emits 186 instructions, each carrying a complete command bundle for one unit plus runtime selectors: row counts may be a constant, the frame count, the context length or the current chunk; bases may be offset by the chunk or the decoder position; key counts by the context length or $\text{pos} + 1$. Control opcodes implement chunk loops (conv1 in 1536-row chunks, FFN in 512-row chunks to fit the hidden bank), the prompt branch, position increment and halt, so the encoder runs for any $n_{\text{frames}} \leq 3000$ and the 92-instruction decoder loop executes once per position.

E. Weights in the source tree

Each tensor is a hex file (one 32-bit word per line) with shape, kind, scales and sha256 in a manifest. At elaboration the generator re-hashes every file and exposes it to a weight store with three backends behind one read-port interface: *RomInit* (a *SyncReadMem* initialised by *\$readmemh* from an image derived at elaboration, the simulation default), *RomLiteral* (a *VecInit* of literals, a true synthesisable ROM) and *SRAM* (a writable memory with a load port). An equivalence test reads every address of real tensors through all three and checks identical data; the literal path is additionally proven bit-exact on three encoder tensors of up to 4.7 Mbit.

V. VERIFICATION

Table II summarises the ladder. Unit vectors are real activations dumped by the golden model on a 2 s clip: the

TABLE III
END-TO-END RUNS ON VERILATOR, 3000 FRAMES EACH.

Set	Clips	= golden	RTL WER	fp32 WER
smoke (2 s; 1024 and 3000 fr.)	1	2/2	0 %	0 %
default (2/5/12 s + 10 utts)	13	13/13	12.75 % [†]	12.75 %
long (20 utts + 29 s clip)	21	21/21	7.58 % [†]	8.08 %

[†] on the LibriSpeech subset; the 29 s clip generates 89 tokens over 60 M cycles.

engine spec covers ten random shapes plus every real weight shape from conv1 (im2col 288 $\rightarrow$ 384) to the LM head (384 $\rightarrow$ 51872, wide mode); the attention spec covers a full 128×128 encoder layer, a ragged 100×70 case with masked keys, and decoder self- and cross-attention at position 41; a dedicated spec covers the engine-to-cache write path. All Verilator runs carry assertions (irrevocable handshakes, sinks never stall, rows, tiles and addresses in range, no same-bank port conflicts, token ids below the vocabulary size) and a requant saturation counter is exposed as a register.

a) *Full-context runs*: Every clip runs at the full 3000-frame window: Whisper-tiny repeats itself when the encoder context is shorter than it was trained on (with a 5 s pad the WER on short clips is 179 %, and the fp32 model behaves the same, decision 12). Runs took 54 min per clip on four Verilator threads and were parallelised one container per clip in the cloud with the same image and the committed weights. Table III gives the results. Where the RTL text differs from the fp32 text (“Hello Bertie” vs. “Hey Bertie”) the golden integer model differs in exactly the same way: the difference is quantisation, not hardware.

b) *Locating chip-level bugs*: A debug spec pauses the sequencer at chosen instructions, dumps the activation banks, and a script diffs them against golden dumps of the same tensors, turning a wrong token into a wrong tensor by binary search over the micro-program. This found four chip-level bugs that the unit tests could not: a GELU fused into the conv2 add that was never applied; a KV transposer that stalled the engine; the KV key index using the row-factor row instead of the tensor row; and firtool silently dropping byte masks on a memory with one unmasked port, now a design rule (decision 13). Unit-level testing had earlier caught an extra quotient bit in the divider (attention outputs exactly $2\times$), a half-word misalignment in the vector unit, a valid-chain reset skew and a 16-bit overflow of the softmax rescale. Each fix is covered by a test that fails without it.

VI. PERFORMANCE

A. Cycle counts and analytic model

A cycle model sums, over the micro-program, the engine formula $K_T N_T (M + 4) + 10$, the per-row vector costs, and an attention formula $(1 + 4(\text{rows} + 4) + 13)$ per engine job, 12 cycles per query row of softmax, 31 per query row of finalisation), all measured at unit level. Table IV shows that it predicts every end-to-end run within 0.2 %; cycle counts are deterministic between the local and cloud runs. At 3000 frames the encoder layers take 37.2 M cycles (88 %), of which

TABLE IV
MEASURED CYCLES VS. THE ANALYTIC MODEL. NO CONSTANT WAS FITTED TO END-TO-END RUNS.

Clip	Frames	Tokens	Measured	Model	Ratio
2 s	1024	6	10,153,984	10,176,473	1.002
2 s	3000	6	42,135,552	42,175,157	1.001
3.6 s utt.	3000	12	43,401,216	43,444,109	1.001
12 s	3000	55	52,477,952	52,538,265	1.001
29 s	3000	89	59,731,968	59,808,337	1.001

TABLE V
LOGIC SYNTHESIS ON ASAP7 (7.5-TRACK RVT, TT CORNER) WITH YOSYS 0.33/ABC, 1,000 ps DELAY TARGET; MEMORIES BLACK-BOXED. CRITICAL PATH IS ABC'S POST-MAPPING REGISTER-TO-REGISTER DELAY BEFORE SETUP AND CLOCK-TO-Q (ABOUT 60 ps IN ASAP7).

Block	Cells	Flops	Area (mm ²)	Crit. path (ps)
Systolic array (32×32 MAC, 8 rows/stage)	706,724	29,477	0.076	1207
variant: 4 rows/stage (not in total)	759,002	41,389	0.083	1100
Matmul engine (array + requant + control)	1,118,387	50,626	0.120	1267
Vector unit (16 lanes)	505,826	1,633	0.050	4843
Attention (softmax, divider)	634,784	13,501	0.059	5824
KV cache logic (transposer)	87,073	16,494	0.010	725
Sequencer	4,860	602	0.000	981
Sampler	8,821	385	0.001	977
Weight store mux (247 banks)	513,535	425	0.040	1027

attention is 23.1 M, or 55 % of the whole run; conv1/conv2 1.2 M; the final LayerNorm and cross-attention K/V 1.9 M; the decoder about 200 k cycles per generated token, of which the LM head is 97 k. Over a whole 3000-frame run the engine is busy 62 % of cycles (26,213,784 of 42,135,552, measured by an on-chip counter); the remainder is softmax and vector phases during which it idles. Encoder attention dominates because 1500 queries × 1500 keys × 6 heads × 4 layers pass through one 32-column array and one 16-lane softmax.

B. ASAP7 synthesis at 1 GHz

To answer what the design costs on a 7 nm process we synthesised the compute blocks with Yosys [9] and ABC onto the ASAP7 predictive PDK [10] (7.5-track RVT cells, typical corner) with a 1 ns delay target, black-boxing the memories; the Verilog is the same firtool output that runs under Verilator, emitted without packed arrays. Table V lists cell count, flop count, standard-cell area and critical path per block. Three blocks meet the target once their serial reductions were replaced by trees (Section IV; the sampler's original 31-deep compare chain synthesised to 9.5 ns): the sequencer, the sampler and the KV-cache logic close at or under 1 ns, and the 247-bank weight-store read mux at 1.03 ns. The systolic array closes at 1.21 ns with eight combinational MAC rows per stage; halving the chain to four rows per stage (a one-line generator parameter, verified bit-exact by the same engine tests) gives 1.10 ns for 10 % more area, which shows that about half of the array's path is the int8 multiplier and 32-bit accumulate rather than the chain. Two blocks are far from 1 GHz and are the honest headline of this table: the attention unit's online-softmax step evaluates, in one FSM cycle, a 64-way maximum tree, a subtraction, the per-head scale multiply, a barrel shift, the exponential table and the rescale multiply (5.8 ns, 166 logic levels), and the vector

TABLE VI
LATENCY FROM THE MEASURED CYCLE COUNTS AT THREE CLOCKS. ONLY THE FIRST COLUMN IS A SYNTHESISED RESULT; THE OTHERS STATE WHAT THE SAME CYCLE COUNTS BUY AT A HIGHER CLOCK.

	as synthesised 0.17 GHz	chains pipelined 0.75 GHz	target 1 GHz
30 s window (42.1 M cycles)	248 ms	56 ms	42 ms
per generated token (200 k)	1.2 ms	0.27 ms	0.20 ms
10 s utterance, 30 tokens	283 ms	64 ms	48 ms
real-time factor	9×10^{-3}	2×10^{-3}	2×10^{-3}

unit's LayerNorm normalisation chains the inverse-square-root Newton step's three multiplies (4.8 ns). Both are single-cycle arithmetic chains inside step-sequenced state machines, so they pipeline mechanically: splitting the softmax step into five stages costs four cycles per query row per key tile, which the cycle model prices at 3.5 M cycles (8 %) on a 3000-frame clip, and the Newton step is executed once per row so its pipeline costs nothing measurable. We did not make these changes for this paper; the figures below therefore state the design's clock as synthesised (0.17 GHz) and, separately, what the 1 GHz cycle counts would deliver once the two chains are pipelined.

a) *Memories*: The on-chip memories are 119 Mbit of SRAM (activation banks 59 Mbit, KV cache 58 Mbit, accumulator 1.6 Mbit, attention and row buffers 0.6 Mbit) and 308 Mbit of weight ROM. ASAP7 ships no memory compiler, so we estimate with published 7 nm densities: a high-density SRAM bit cell of $0.027 \mu\text{m}^2$ [11] yields, at a macro efficiency of 50 %, about $0.054 \mu\text{m}^2/\text{bit}$, i.e. 6.4 mm^2 for the SRAM; a via-programmed ROM at roughly $0.01 \mu\text{m}^2/\text{bit}$ yields 3 mm^2 for the weights, or 6.4 mm^2 more if the weights are instead held in SRAM to keep them swappable. The chip is therefore memory-dominated: roughly 10–13 mm^2 of memory against 0.28 mm^2 of logic, before routing overhead and I/O.

b) *Latency and throughput*: Cycle counts are exact, so latency is one division away once a clock is fixed. Table VI gives the three that matter: the design as synthesised (0.17 GHz, attention-limited), the clock it reaches once the two chains of Section VI-B are pipelined (0.75 GHz, then engine-limited), and the 1 GHz target. At 1 GHz a 30 s window costs 42.1 ms plus 0.2 ms per generated token, so a typical 10 s utterance of 30 tokens is transcribed in 48 ms, a real-time factor of 5×10^{-3} , and one engine sustains about 17 windows per second, or 500 s of audio per second of wall time. Because the encoder runs for any $n_{\text{frames}} \leq 3000$, a host that pads to the true speech length rather than 30 s would cut the encoder cost roughly in proportion, at the accuracy cost documented in decision 12.

c) *Power*: We have no switching-activity simulation, so we give only a first-order bound. With the array 70 % busy and 50 fJ per int8 MAC including its 32-bit accumulate (a mid-range figure for 7 nm int8 arrays), the 1024 MACs draw about 36 mW at 1 GHz (the clock the design would reach after the pipelining above, not as synthesised); a 2048-bit weight-tile word every four cycles and a 256-bit activation word per cycle at 20 fJ/bit add about 15 mW, and the remaining logic and

clock tree perhaps as much again. A total of order 100 mW at 1 GHz, i.e. about 5 mJ per 30 s window, is our estimate; it is not a measurement.

VII. RELATED WORK

Integer inference for transformers is well studied in software: LLM.int8 [3] identified the outlier channels that force our int16 residual, and SmoothQuant [4] migrates them into the weights; we implemented the latter but did not need it. Online softmax [5] and FlashAttention [6] motivate the 64-key tiled attention. On the hardware side, Chisel [7] generators of ML accelerators such as Gemmini [8] provide a systolic array and a software stack; whisper-si differs in scope (one fixed model, whole graph and weights on chip, micro-program generated from the graph) and in its verification contract (bit-exactness against a golden model on real audio rather than functional tests). Whisper-specific hardware work has so far targeted GPUs, NPU or FPGAs with floating-point or mixed-precision arithmetic; to our knowledge this is the first fully integer, bit-exact Whisper implementation in RTL.

VIII. LIMITATIONS AND FUTURE WORK

Accuracy runs use the full 30 s window rather than a speech-length context; calibration used dev-clean only, and the requant saturation counter is the only runtime guard on unseen inputs; the two end-to-end sets overlap in ten utterances, so 24 distinct clips ran on the RTL. The synthesis numbers are pre-layout, come from an open-source mapper rather than a signoff tool, and exclude the memories, whose area is estimated from published densities; no place-and-route or power simulation was run, and wire load is not modelled, so the reported paths are optimistic. Reaching 1 GHz needs three pipeline changes we did not make here: the attention softmax step, the LayerNorm Newton chain, and the matmul drain's requant multiply. The performance steps beyond that are a second softmax pipeline overlapping the *PV* of one tile with the exponentials of the next, and letting the vector unit run concurrently with the engine on independent rows; the analytic model sizes all of them before they are built.

IX. REPRODUCING

make setup, refs-check, weights-check, golden, test (18 Python and 49 Verilator tests, 18 min), e2e-smoke (one clip on the full chip), e2e-modal (both sets, one container per clip) and report (per-clip tables and the cycle model) reproduce every figure above; sbt "Test/runMain whisper.synth.EmitSynth" emits the synthesis Verilog.

X. CONCLUSION

A small, fully integer speech recogniser can be specified to the bit in a few hundred lines of numpy, generated in Chisel with its weights as ordinary source files, and shown to reproduce that specification exactly on real audio. The cost of the approach is discipline (no tolerances, every rounding explicit); its reward is that every bug found at chip level was a hardware bug with

a numerical fingerprint, and that the accelerator's accuracy is a property of the Python model, settled before the RTL was written.

REFERENCES

- [1] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proc. ICML*, 2023.
- [2] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, "LibriSpeech: An ASR corpus based on public domain audio books," in *Proc. ICASSP*, 2015.
- [3] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," in *Proc. NeurIPS*, 2022.
- [4] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and efficient post-training quantization for large language models," in *Proc. ICML*, 2023.
- [5] M. Milakov and N. Gimelshein, "Online normalizer calculation for softmax," arXiv:1805.02867, 2018.
- [6] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Proc. NeurIPS*, 2022.
- [7] J. Bachrach *et al.*, "Chisel: Constructing hardware in a Scala embedded language," in *Proc. DAC*, 2012.
- [8] H. Genc *et al.*, "Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration," in *Proc. DAC*, 2021.
- [9] C. Wolf, "Yosys open synthesis suite," <https://yosyshq.net/yosys/>.
- [10] L. T. Clark *et al.*, "ASAP7: A 7-nm finFET predictive process design kit," *Microelectronics Journal*, vol. 53, pp. 105–115, 2016.
- [11] J. Chang *et al.*, "A 7nm 256Mb SRAM in high-k metal-gate FinFET technology with write-assist circuitry for low- V_{MIN} applications," in *ISSCC Dig. Tech. Papers*, 2017.